

Побудова генетичного алгоритму

Завдання

Завдання: Запрограмувати вирішення обраної задачі за допомогою генетичного алгоритму. Дати можливість обирати кількість та якість початкової популяції чи декількох популяцій. Кількість ітерацій має теж задаватись, включно з опцією “до останнього живого”. Виводити статистику популяцій після кожної ітерації.

Види генетичних алгоритмів, які треба застосувати. Всі непарні номери по списку - перший варіант, всі парні - другий. На кожен варіант є 2 пропозиції генетичного алгоритму, можна брати любую. За вибір другої, яка трохи складніша - +2 порівняно з першою.

1.1) Звичайний генетичний алгоритм (Simple Genetic Algorithm): Заснований на базових операціях кросоверу, мутації та вибору. (5 балів)

1.2) Мультимодальний генетичний алгоритм (Multimodal Genetic Algorithm): Призначений для пошуку різних локальних мінімумів або максимумів, що дозволяє знаходити різноманітні оптимальні рішення. (7 балів)

2.1) Елітний генетичний алгоритм (Elitist Genetic Algorithm): Використовує додатковий елітарний механізм, що дозволяє зберігати найкращі особини у кожній популяції. (5 балів)

2.2) Паралельний генетичний алгоритм (Parallel Genetic Algorithm): Використовує паралельні обчислення для прискорення обробки великих популяцій та задач з високою обчислювальною складністю. (7 балів)

Обов'язкові Етапи

У кожному алгоритмі мають бути наступні кроки. Коментарями в коді їх треба підписати. Найкраще, щоб (майже) кожен крок був в окремому методі з відповідною назвою, навіть якщо він займає 1 строчку (проситиму пояснити, чому так зручно та чому це хороша практика).

1) **Ініціалізація популяції:** Створення початкової популяції із випадковими особинами (кандидатами на рішення).

2) **Оцінка придатності** (fitness evaluation): Обчислення значень придатності для кожної особини у популяції на основі поставленої задачі. (ТЕСТИ до цього функціоналу)

3) **Вибір батьків**: Вибір особин для формування нащадків (на основі їхньої придатності). Можливі різні методи вибору, такі як випадковий вибір, вибір турнірний, вибір за рулеткою тощо. (ТЕСТ до цього функціоналу)

4) **Створення нащадків**: Застосування генетичних операторів (кросоверу та мутації) до обраних батьків для створення нової популяції нащадків. (ТЕСТИ до цього функціоналу)

5) **Оцінка придатності нової популяції**: Обчислення значень придатності для кожної особини у новій популяції. (ТЕСТИ до цього функціоналу)

6) **Вибір для наступної популяції**: Вибір особин для наступної популяції, як правило, здійснюється за принципом елітарності (збереження найкращих особин) та може включати також нових нащадків. (ТЕСТИ до цього функціоналу)

7) **Умова зупинки**: Перевірка умови зупинки, такої як досягнення заданої кількості ітерацій або зміна величини придатності менше за певний поріг.

Повторення: Повторення кроків 3-7, доки не буде виконана умова зупинки.

8) **Вивід результатів**: Повернення найкращого знайденого рішення або найкращої особини у популяції. (ТЕСТИ до цього функціоналу)

Тести

Тести необхідні, написала, до яких кроків їх застосовувати - 5 балів. Дуже якісні та на всі випадки життя тести - 7 балів. Непротестований алгоритм, але наявні тести - менше 5 балів.

Додаткові бали

Додаткові бали можна заробити за графічну візуалізацію процесу, а також за додаткові використані технології: Web-UI, DB, Docker, використання багатопоточності та ін. Одна технологія, повноцінно та доречно застосована - 5 балів, мінімальне застосування - 2-3 бали, дуже якісне застосування, пояснення високої ідеї та подібне - 7 балів.

Також додаткові бали нараховуються за якісне виконання не просто генетичного алгоритму, а за вирішення з його допомогою однієї з задач, наведених нижче.

Етапи рішення задачі мають візуалізуватись (графічно або цифрено), щоб було зрозуміло, як що покращується з кожною ітерацією.

Задача комівояжера (Travelling Salesman Problem): Мінімізація відстані, яку потрібно подолати продавцеві, щоб відвідати кожного клієнта точно один раз і повернутися в точку виходу.

Задача планування (Scheduling Problem): Оптимізація розподілу завдань або подій в часі для мінімізації загального часу виконання або максимізації використання ресурсів.

Задача побудови розкладу (Timetabling Problem): Розподіл ресурсів, таких як кімнати або обладнання, для заданих подій або класів з метою мінімізації конфліктів або максимізації використання.

Задача оптимізації параметрів: Пошук оптимальних параметрів для моделей машинного навчання, нейронних мереж або інших алгоритмів, що вимагають налаштування параметрів.

Задача розташування (Placement Problem): Оптимізація розташування об'єктів або ресурсів на площині з метою мінімізації перекриття або максимізації покриття.

Питання (до 3х балів)

Необхідно вміти пояснити, що таке генетичний алгоритм, який клас задач він вирішує та чому такі задачі відносяться до кіберфізичних систем.